Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

CS5741
Concurrency & Parallelism in Software Development

Lecture 3 – Parallel Algorithms

Dr Faeq Alrimawi

HOST INSTITUTION
UNIVERSITY OF LIMERICK
OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS
NUI Galway OÉ Gaillimh
Maynooth University
Trinity College Dublin
University College Dublin
UCC
MTU
DCU
Waterford Institute of Technology
Dundalk Institute of Technology
LIT
GMIT

Recap... Identifying Parallelisable Problems

6

What Are Parallelisable Problems?

- Those that can be divided into smaller, independent tasks that can be executed simultaneously
  - Sub-tasks do not interfere with each other
  - Allows for parallel execution across multiple processors or cores
- Example: Applying filter (e.g., greyscale) to an image



11

Parallelisable Problem Characteristics

- Decomposability
  - The problem can be divided into smaller chunks (sub-problems).
  - Example: Matrix multiplication where each element of the result matrix can be computed separately.
- Independence
  - Tasks should be independent or have minimal dependencies.
  - Example: Image processing tasks where each pixel can be processed independently.
- Granularity
  - The size of the tasks should be large enough to justify the overhead of parallel execution.
  - Example: Sorting large arrays versus small arrays.

29

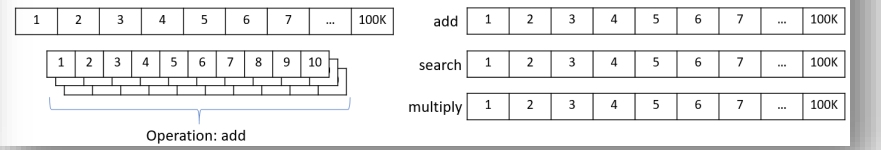
Decomposition Analysis

- Breaking down a problem into smaller, manageable sub-problems that can be executed concurrently or in parallel.
- Key Steps:
  - Components Analysis: determine the major components of the problem.
  - Dependency Analysis: ensure tasks are independent or have minimal dependencies.
  - Communication Analysis: analyse the communication requirements between tasks

35

Data Parallelism vs. Task Parallelism

- Two paradigms to divide the problem.
- Data parallelism focuses on dividing the data
- Task parallelism focuses on dividing the tasks (functions/operations)



Today, We Are Covering…

- Parallel Algorithms: Intro
- Key Concepts
- Parallel Algorithm Design Methodology
- Common Parallel Algorithm Patterns
- Activity: Designing a Parallel Algorithm

Parallel Algorithms: Intro (1-2)

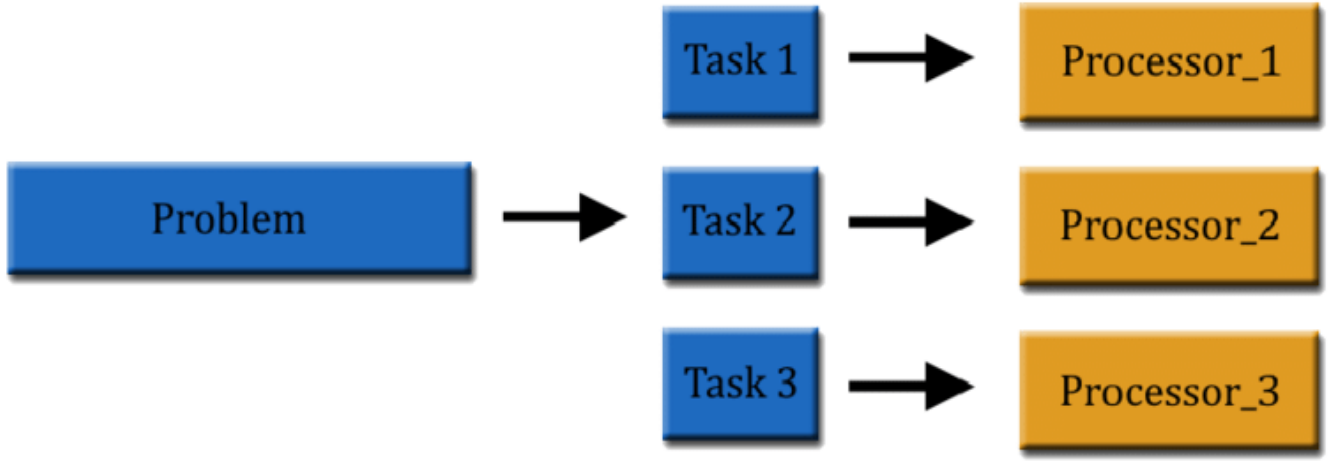
- Parallel algorithms are designed to **execute multiple computations simultaneously**.
  - Utilising parallel computing resources to solve problems more efficiently than serial algorithms.
- Essential for **high-performance applications** in areas
  - Such as scientific computing, big data processing, and real-time systems.
- Necessary for **leveraging multi-core processors** and distributed computing systems.

Parallel Algorithms: Intro (2-2)

Serial Algorithm



Parallel Algorithm



Code Example: Serial Sum

```
private static void runSequential(int [] array) {

    long startTime = System.nanoTime();

    long sum = 0;

    for (int value : array) {
        sum += value;
    }

    long endTime = System.nanoTime();

    System.out.println("Sequential Sum: " + sum);

    System.out.println("Time taken: " + (endTime - startTime) / 1_000_000 + " ms");
}
```

Code Example: Parallel Sum

```java
//****Try changing the size of the array
int arraySize = 100_000_000;
int[] array = generateArray(arraySize);

ForkJoinPool pool = new ForkJoinPool();

long startTime = System.nanoTime();

SumTask task = new SumTask(array, start:0, array.length);
long sum = pool.invoke(task);

long endTime = System.nanoTime();
System.out.println("Parallel Sum: " + sum);
System.out.println("Time taken: " + (endTime - startTime) / 1_000_000 + " ms");
```

```java
private static class SumTask extends RecursiveTask<Long> {
    private final int[] array;
    private final int start;
    private final int end;

    //**** try changing this value to see the impact
    private static final int THRESHOLD = 100_000;

    public SumTask(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

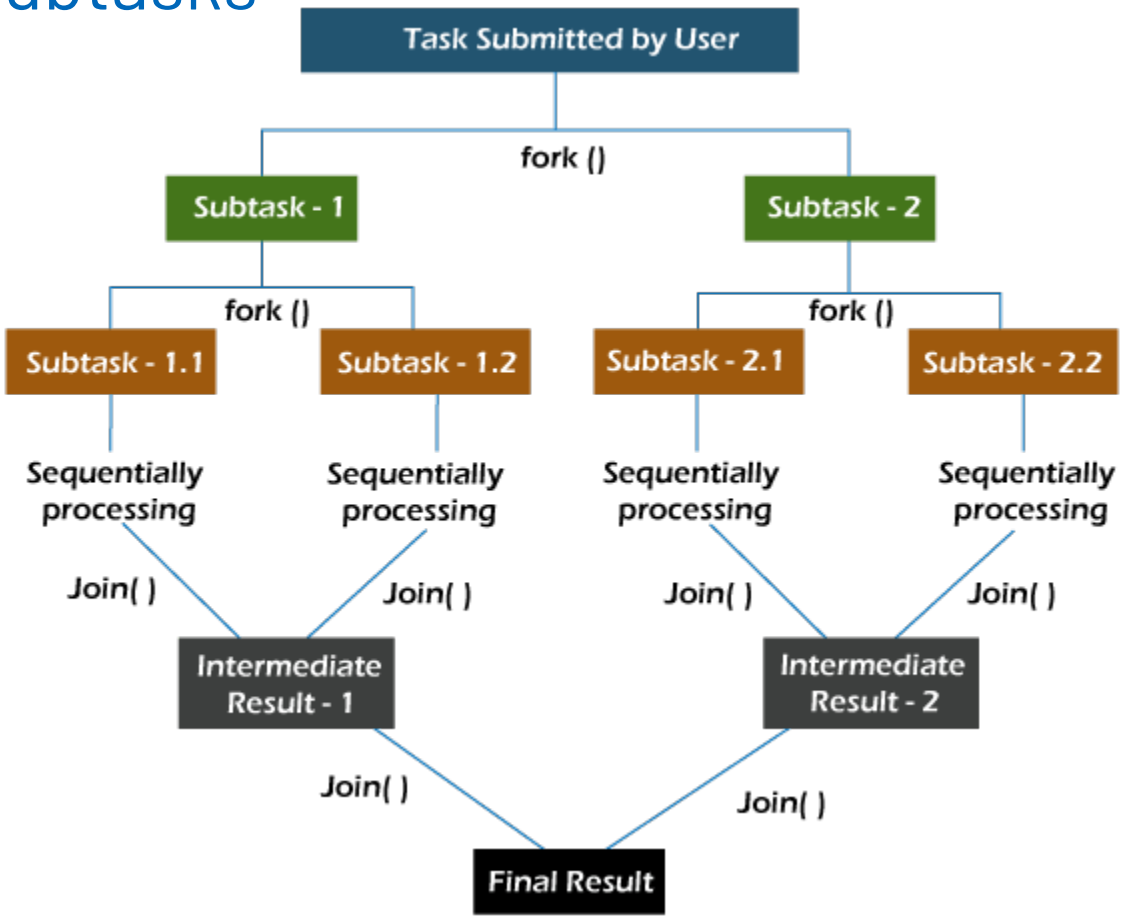
    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) {
                sum += array[i];
            }
            return sum;
        } else {
            int mid = (start + end) / 2;
            SumTask leftTask = new SumTask(array, start, mid);
            SumTask rightTask = new SumTask(array, mid, end);

            leftTask.fork(); // Start left task asynchronously
            long rightResult = rightTask.compute(); // Compute right task synchronously
            long leftResult = leftTask.join(); // Wait for left task and get result

            return leftResult + rightResult;
        }
    }
}
```

ForkJoin Framework

- Divide (fork) a task recursively into subtasks
- Subtasks are further divided into subtasks until computation is small
- Results from each subtask are combined (joined) back
- Divide and Conquer



Key Classes

- ForkJoinPool
  - Manages worker threads
- RecursiveTask (RecursiveAction)
  - Represent unit of work to do
  - **compute()** is key method
    - Determines how big/small sequential execution is, and thus how many tasks
    - You have control over this

```java
private static class SumTask extends RecursiveTask<Long> {
    private final int[] array;
    private final int start;
    private final int end;

    //**** try changing this value to see the impact
    private static final int THRESHOLD = 100_000;

    public SumTask(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) {
                sum += array[i];
            }
            return sum;
        } else {
            int mid = (start + end) / 2;
            SumTask leftTask = new SumTask(array, start, mid);
            SumTask rightTask = new SumTask(array, mid, end);

            invokeAll(leftTask, rightTask);
            long rightResult = rightTask.join(); //or get()
            long leftResult = leftTask.join();

            return leftResult + rightResult;
        }
    }
}
```

Code Example: Parallel Sum

```java
//****Try changing the size of the array
int arraySize = 100_000_000;
int[] array = generateArray(arraySize);

ForkJoinPool pool = new ForkJoinPool();

long startTime = System.nanoTime();

SumTask task = new SumTask(array, start:0, array.length);
long sum = pool.invoke(task);

long endTime = System.nanoTime();
System.out.println("Parallel Sum: " + sum);
System.out.println("Time taken: " + (endTime - startTime) / 1_000_000 + " ms");
```

Serial Fraction?

THRESHOLD/arraySize =
100,000/100,000,000
=
0.001 -> 0.1% (99.9% parallel, Ideally!!)

```java
private static class SumTask extends RecursiveTask<Long> {
    private final int[] array;
    private final int start;
    private final int end;

    //**** try changing this value to see the impact
    private static final int THRESHOLD = 100_000;

    public SumTask(int[] array, int start, int end) {
        this.array = array;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            long sum = 0;
            for (int i = start; i < end; i++) {
                sum += array[i];
            }
            return sum;
        } else {
            int mid = (start + end) / 2;
            SumTask leftTask = new SumTask(array, start, mid);
            SumTask rightTask = new SumTask(array, mid, end);

            leftTask.fork(); // Start left task asynchronously
            long rightResult = rightTask.compute(); // Compute right task synchronously
            long leftResult = leftTask.join(); // Wait for left task and get result

            return leftResult + rightResult;
        }
    }
}
```

Today, We Are Covering…

- Parallel Algorithms: Intro
- Key Concepts
- Parallel Algorithm Design Methodology
- Common Parallel Algorithm Patterns
- Activity: Designing a Parallel Algorithm

Key Concepts

Task Decomposition

- Dividing the problem into smaller tasks that can be executed concurrently.
- Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).

Task Dependency

- Understanding dependencies between tasks to avoid incorrect execution.
- Dependencies: Data dependencies, control dependencies, and resource dependencies.

Load Balancing

- Evenly distributing work across processors to avoid idle time.
- Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).

Synchronisation

- Ensuring that tasks operate correctly when they need to access shared resources.
- Mechanisms: Locks, barriers, and atomic operations.

Key Concepts

Task Decomposition

- Dividing the problem into smaller tasks that can be executed concurrently.
- Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).

Task Dependency

- Understanding dependencies between tasks to avoid incorrect execution.
- Dependencies: Data dependencies, control dependencies, and resource dependencies.

Load Balancing

- Evenly distributing work across processors to avoid idle time.
- Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).

Synchronisation

- Ensuring that tasks operate correctly when they need to access shared resources.
- Mechanisms: Locks, barriers, and atomic operations.

Task Decomposition

- The primary goal is to **identify as much parallelism as possible** by identifying independent tasks that can be executed simultaneously.
- Data Decomposition (Data Parallelism)
  - **Splits the data set into chunks** that can be processed independently by the same task running in parallel.
  - Example: in matrix multiplication, the matrices can be divided into sub-matrices.
- Functional Decomposition (Task Parallelism)
  - **Divides the problem into distinct tasks** that perform different functions or operations. Each task can be executed in parallel on different data or different parts of the problem.
  - Example: in a video processing pipeline, one task may handle decoding, another handles filtering, and a third handles encoding

Key Concepts

Task Decomposition

- Dividing the problem into smaller tasks that can be executed concurrently.
- Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).

Task Dependencies

- Understanding dependencies between tasks to avoid incorrect execution.
- Dependencies: Data dependencies, control dependencies, and resource dependencies.

Load Balancing

- Evenly distributing work across processors to avoid idle time.
- Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).

Synchronisation

- Ensuring that tasks operate correctly when they need to access shared resources.
- Mechanisms: Locks, barriers, and atomic operations.

Task Dependencies

- The execution of one task depends on the completion of another.
- Data Dependency
  - A task requires data produced by another task. This can be further broken down into:
    - Read-After-Write (RAW) Dependency: Task B cannot *read* data until Task A *writes* it. (true dependency)
    - Write-After-Read (WAR) Dependency: Task B cannot *write* data until Task A *reads* it. (anti-dependency)
    - Write-After-Write (WAW) Dependency: Task B cannot *write* data until Task A *writes* it. (output dependency)
- Control Dependency
  - The **execution order** of tasks is determined by control structures like conditionals or loops.
    - For example, one task may only execute if a certain condition, evaluated by another task, is true.
- Resource Dependency
  - Multiple tasks require access to the same resource, such as memory or I/O device.
  - Can be also broken into: RAW, WAR, and WAW.

Key Concepts

Task Decomposition

- Dividing the problem into smaller tasks that can be executed concurrently.
- Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).

Task Dependencies

- Understanding dependencies between tasks to avoid incorrect execution.
- Dependencies: Data dependencies, control dependencies, and resource dependencies.

Load Balancing

- Evenly distributing work across processors to avoid idle time.
- Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).

Synchronisation

- Ensuring that tasks operate correctly when they need to access shared resources.
- Mechanisms: Locks, barriers, and atomic operations.

Load Balancing

- Distributing the workload evenly across all available processors
  - Ensures that each processor is utilised efficiently and that no processor underutilised or overloaded.
- Static Load Balancing
  - The **workload is divided and assigned to processors at the start**, based on a predefined strategy.
  - Pros: **simple** to implement, **minimal** runtime **overhead**.
  - Cons: **ineffective** if the **workload** is **unpredictable** or if tasks vary significantly in execution time.
  - Example: dividing a large dataset into equal-sized chunks and assigning each chunk to a processor.
- Dynamic Load Balancing
  - The **workload** is **distributed during execution**, based on the current state of each processor.
  - Pros: **more flexible** and **adaptable** to variations in workload.
  - Cons: Adds runtime **overhead** due to task redistribution and scheduling.
  - Example: In a parallel search algorithm, if one processor finishes early, it can take on tasks from another processor that is still busy.

Key Concepts

Task Decomposition

- Dividing the problem into smaller tasks that can be executed concurrently.
- Types: Data Decomposition (splitting data) and Functional Decomposition (splitting functions or tasks).

Task Dependencies

- Understanding dependencies between tasks to avoid incorrect execution.
- Dependencies: Data dependencies, control dependencies, and resource dependencies.

Load Balancing

- Evenly distributing work across processors to avoid idle time.
- Strategies: Static (predefined distribution) vs. Dynamic (load balancing at runtime).

Synchronisation

- Ensuring that tasks operate correctly when they need to access shared resources.
- Mechanisms: Locks, barriers, and atomic operations.

Synchronisation

- Coordinating the execution of tasks in a parallel algorithm. Ensures that:
  - Shared resources are accessed safely, and
  - tasks proceed in the correct order when dependencies exist.
- Prevents issues like race conditions.
- Mechanisms
  - Locks/Mutexes
    - Ensure that only one processor accesses a critical section of code at a time. This prevents data races but can lead to bottlenecks if overused.
    - Example: a mutex might be used in a parallel banking application to ensure that only one transaction updates an account balance at a time.
  - Barriers
    - Synchronises tasks by making them all wait until every task has reached a certain point before any can proceed.
    - Example: matrix multiplication, a barrier can ensure that all sub-matrices are multiplied before the results are summed.

Synchronisation - Problems

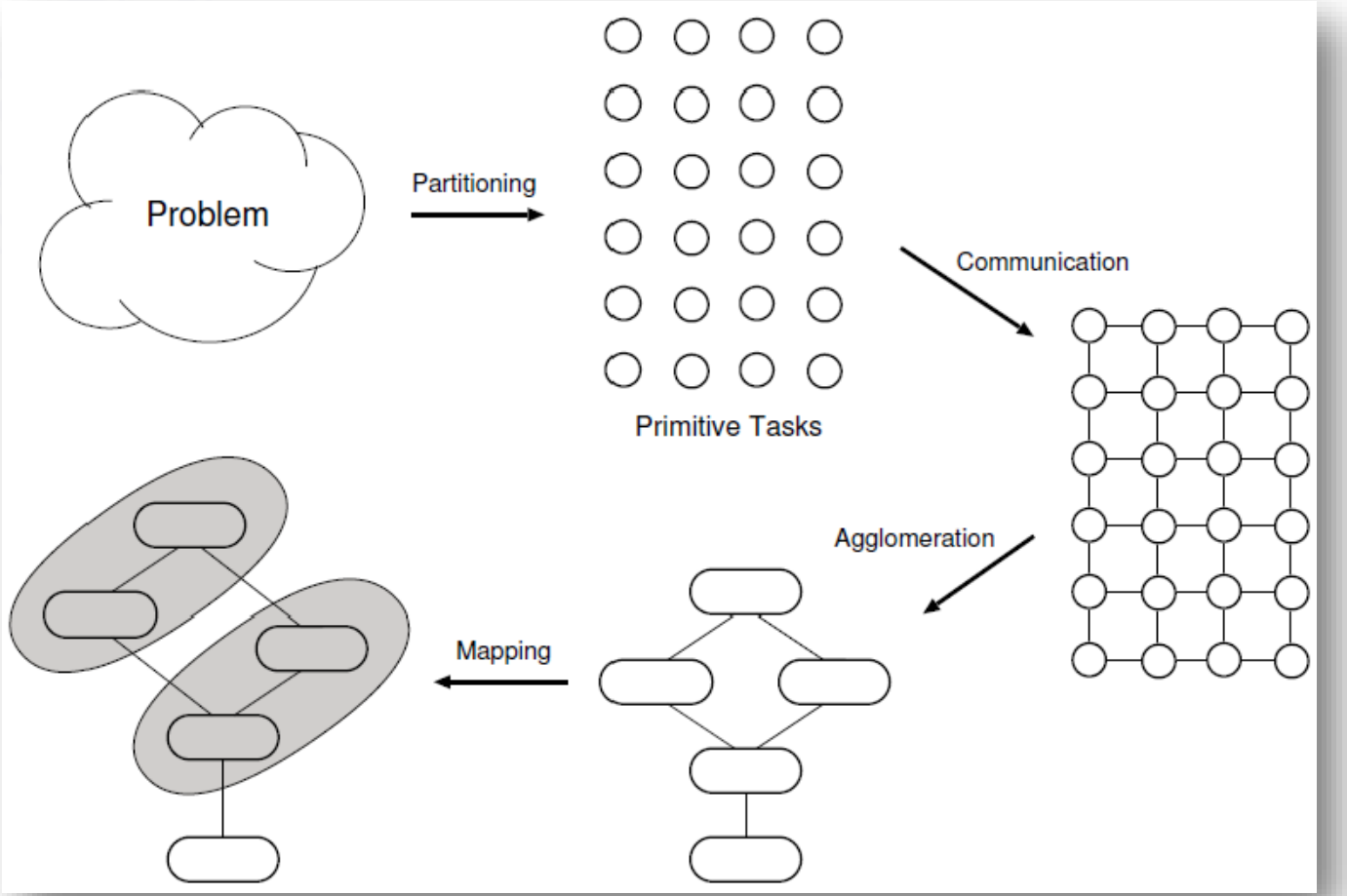
- Deadlocks
  - Occur when two or more tasks wait indefinitely for each other to release resources, leading to a complete halt in execution.
- Livelocks
  - Tasks continuously change their state in response to each other without making progress.
- Overhead
  - Excessive synchronisation can negate the benefits of parallelism by causing tasks to spend more time waiting than executing.

Today, We Are Covering…

- Parallel Algorithms: Intro
- Key Concepts
- **Parallel Algorithm Design Methodology**
- Common Parallel Algorithm Patterns
- Activity: Designing a Parallel Algorithm

Foster's Design Methodology (FDM)

A **four-stage design process** whose input is the problem statement



Foster's Design Methodology (FDM)

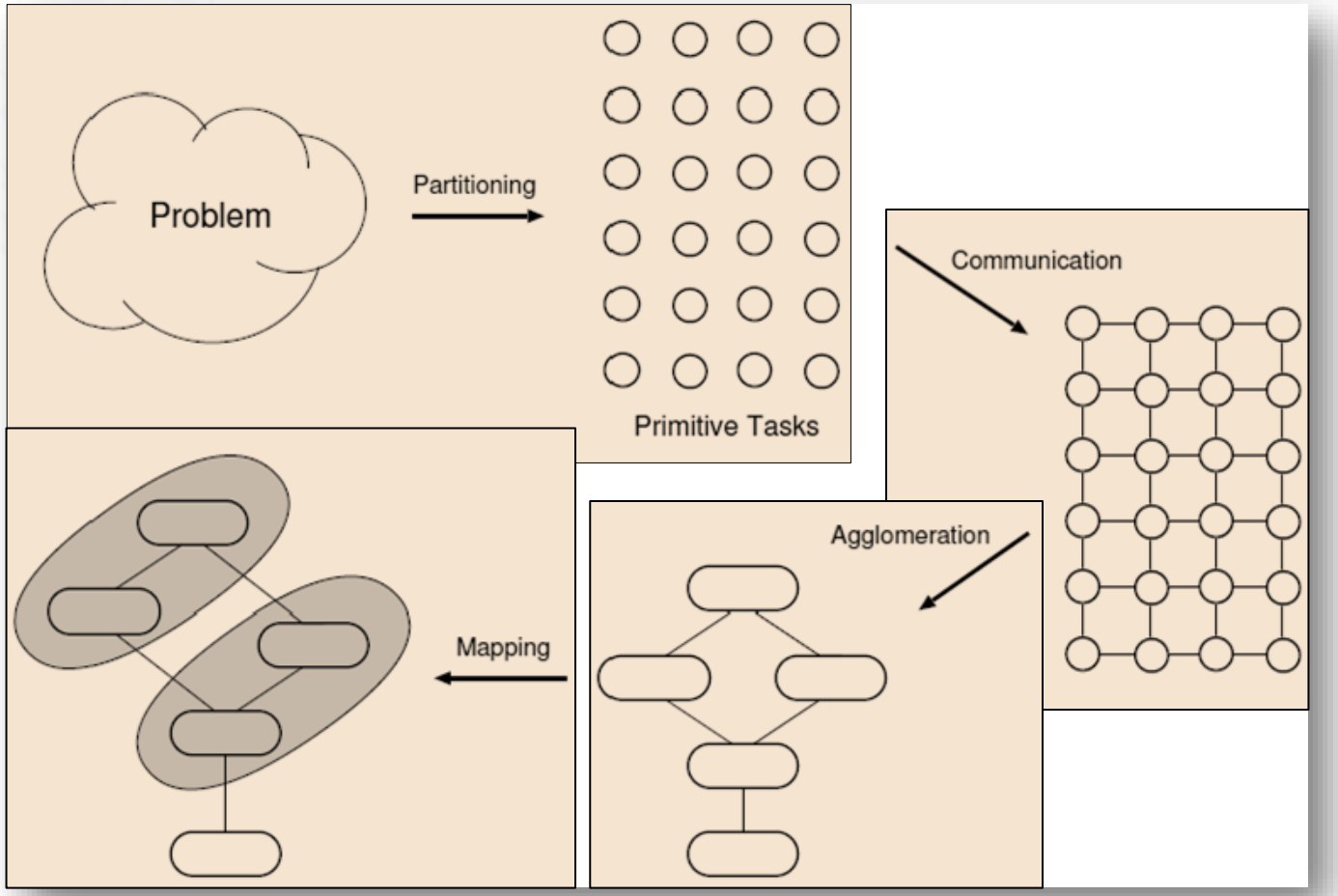
A **four-stage design process** whose input is the problem statement

1 Partitioning: dividing the computation and the data into chunks

2 Communication: determining how tasks will communicate with each other

3 Agglomeration: grouping tasks into larger tasks to improve performance

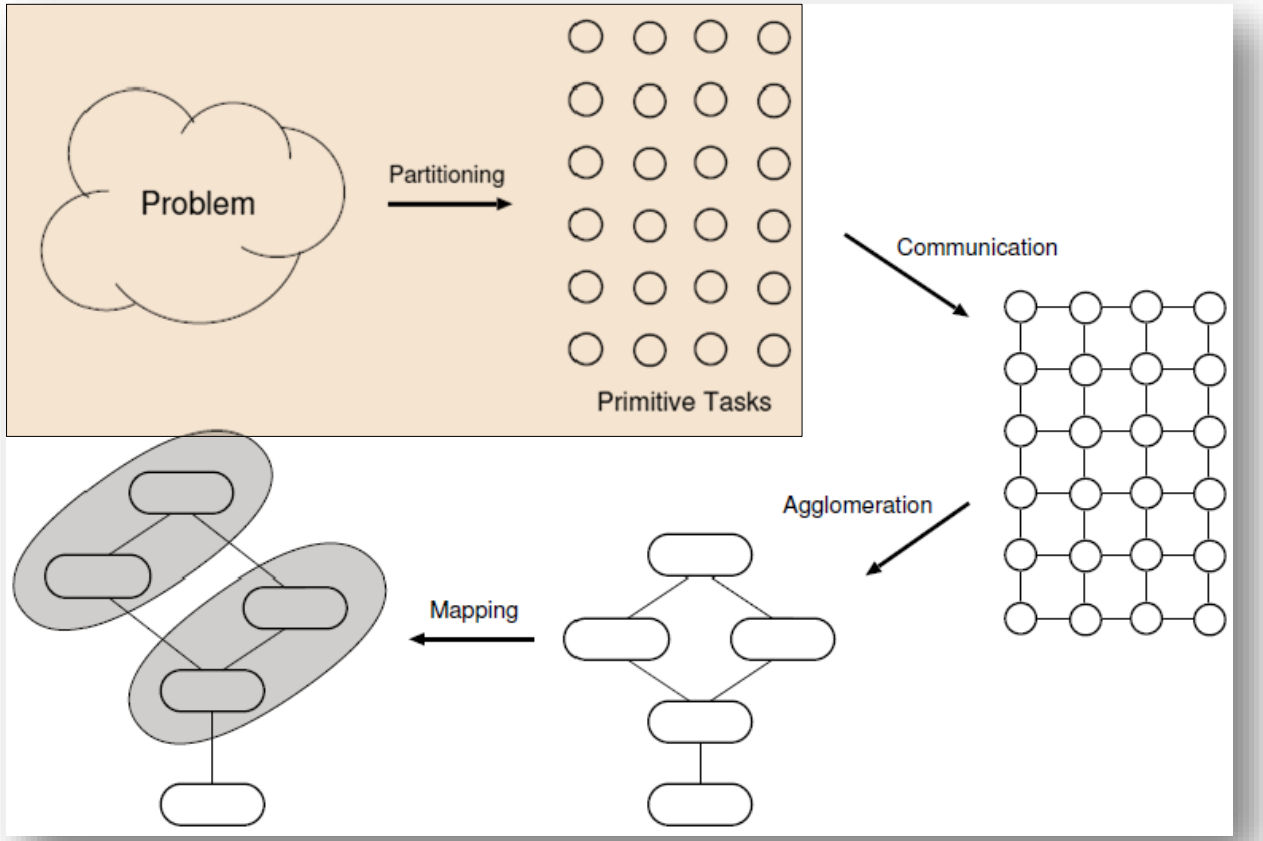
4 Mapping: assigning tasks to physical processors



FDM: Partitioning (1-3)

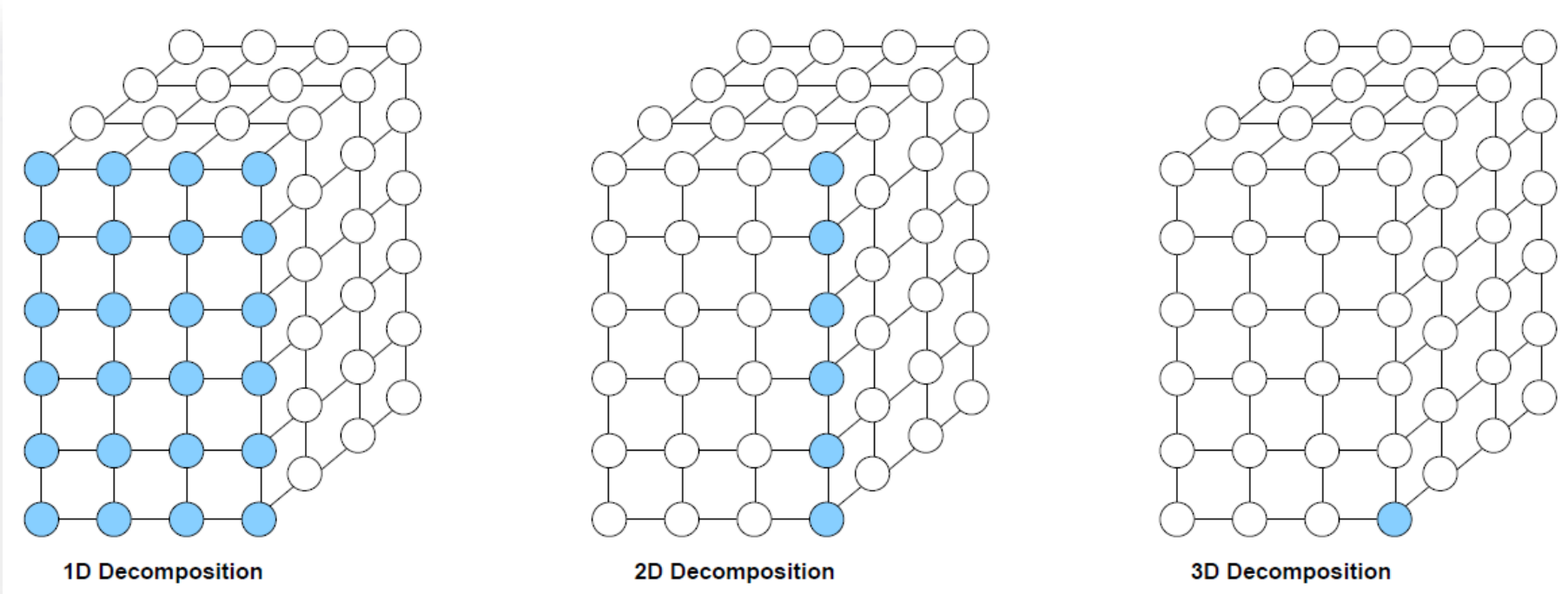
- The goal is to discover as many parallel partitions as possible.

- Methods:
  - Domain (Data) Decomposition
  - Functional (Task) Decomposition



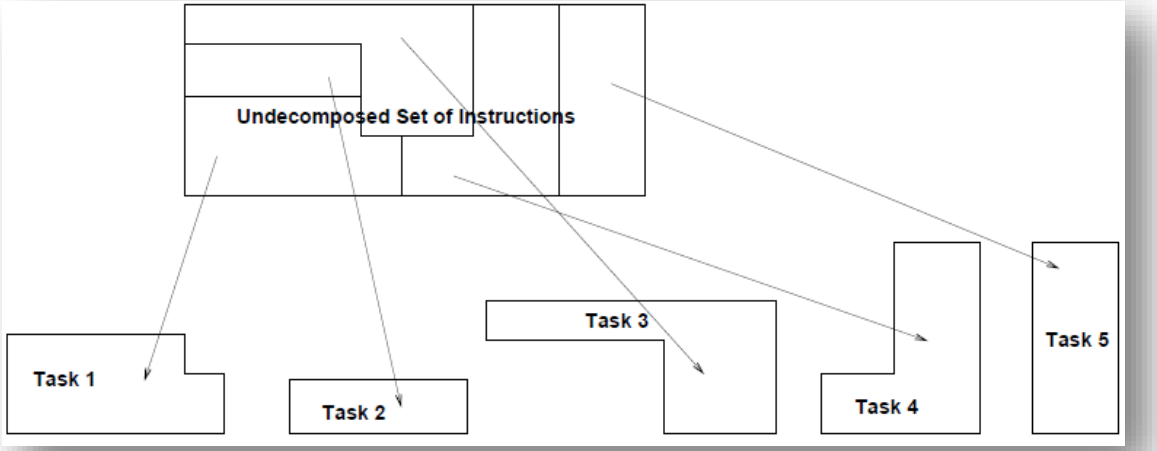
FDM: Partitioning (2-3)

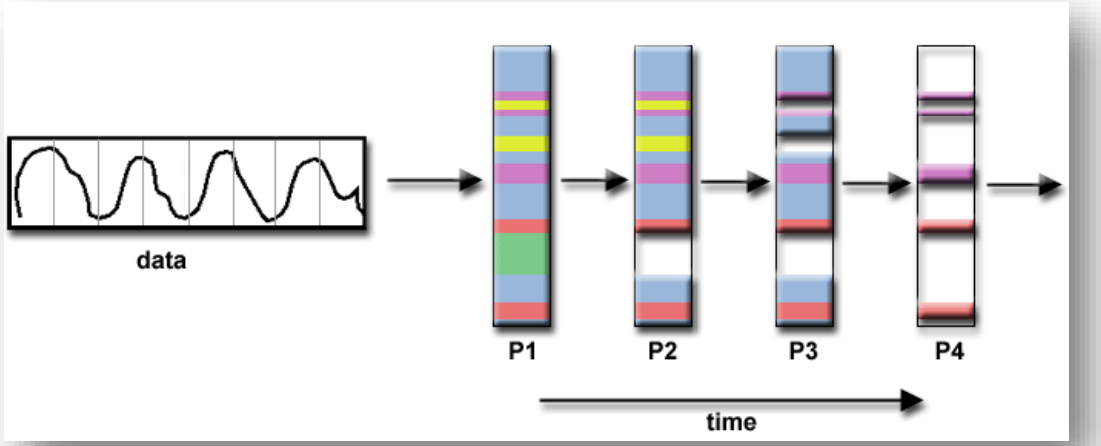
- Domain (Data) Decomposition
  - Split data into parallel chunks



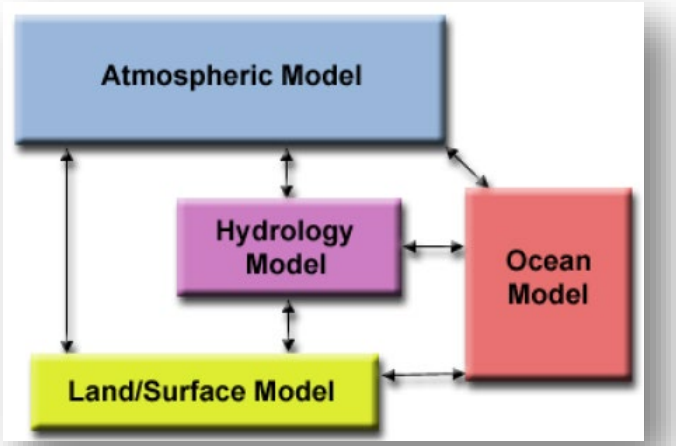
FDM: Partitioning (3-3)

- Functional (Task) Decomposition
  - Decompose problem into parallel functions/tasks.
  - Can be applied to whole or part of the data set.





Pipeline decomposition



Non-Pipeline decomposition

Foster's Checklist for Partitioning

✓The partition defines at least an order of magnitude **more tasks than there are processors** in your target computer.

✓Primitive **tasks** are roughly the **same size**.

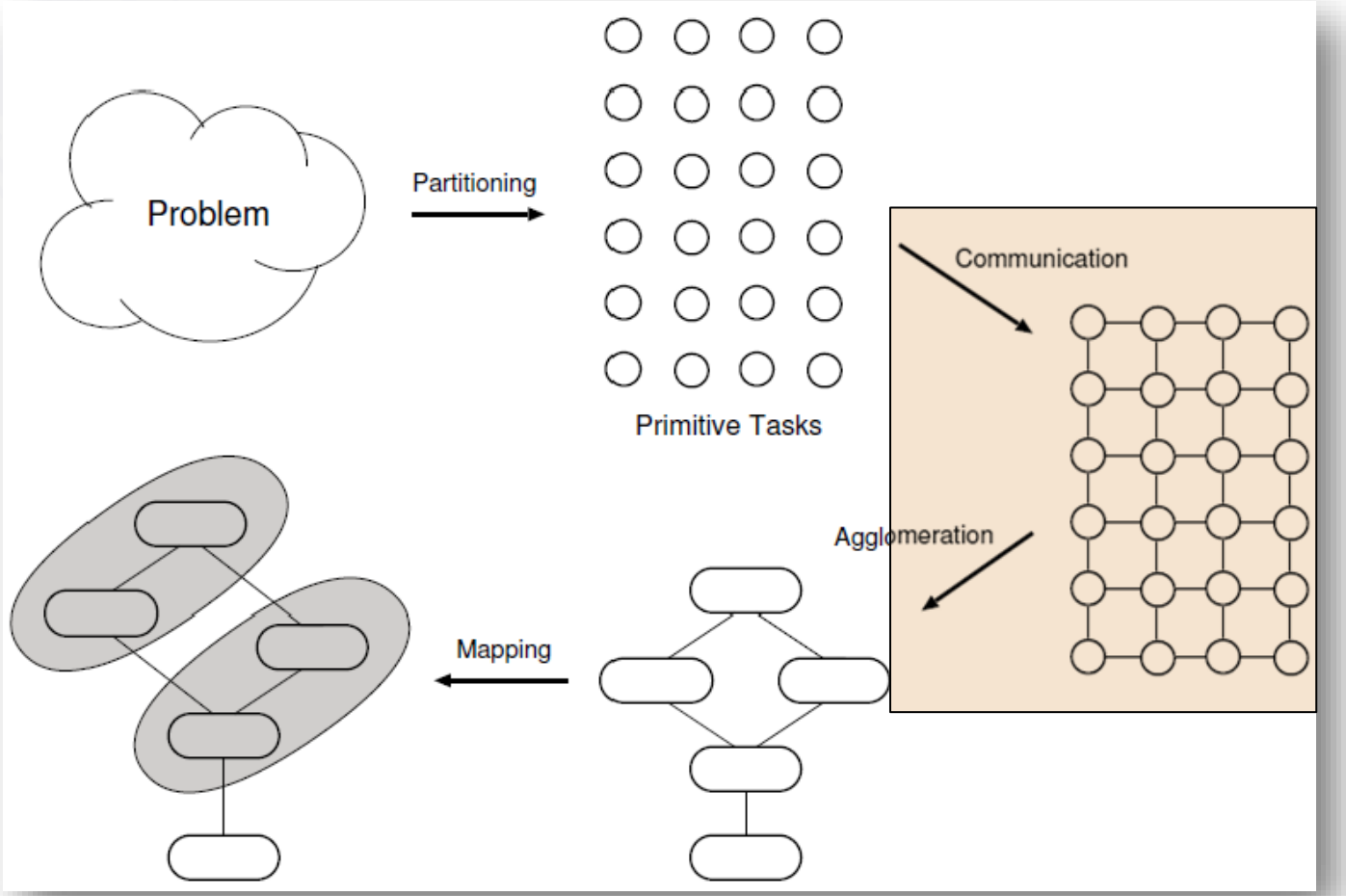
✓The **number** of **tasks** is an **increasing** function of the **problem size**.

- Ideally, an increase in problem size should increase the number of tasks rather than the size of individual tasks.

✓You have identified several **alternative partitions**.

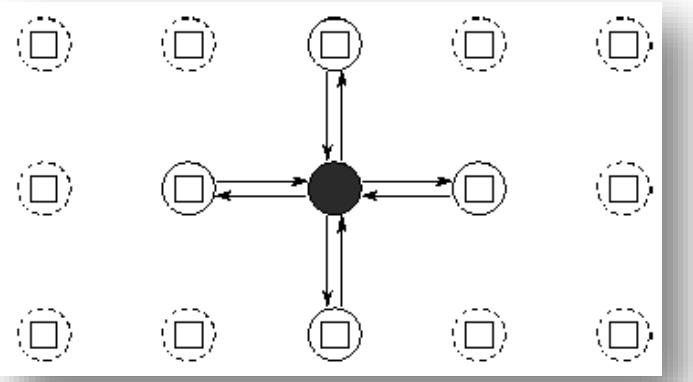
FDM: Communication (1-2)

- Define **data flow** between tasks



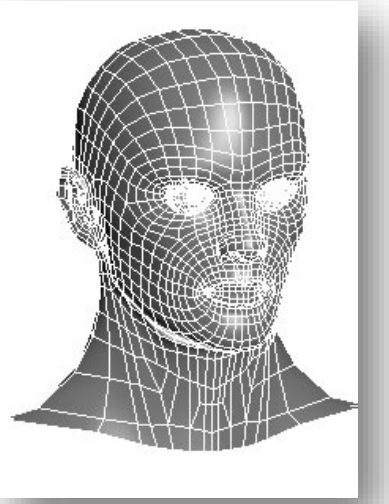
FDM: Communication (2-2)

- Local Communication vs Global Communication
  - Local: when a task needs values from a smaller number of other tasks.
  - Global: when a great many tasks must contribute data to perform a computation.



Structured Communication

- Structured Communication vs Unstructured Communication
  - Structured: regular communication pattern between tasks
  - Unstructured: far irregular communication patterns between tasks



Unstructured Communication

Foster's Checklist for Communication

✓All tasks perform about the **same number of communication operations.**

- Unbalanced communication requirements suggest a design that will not scale.

✓Each task should communicate **only with a small number of neighbours.**

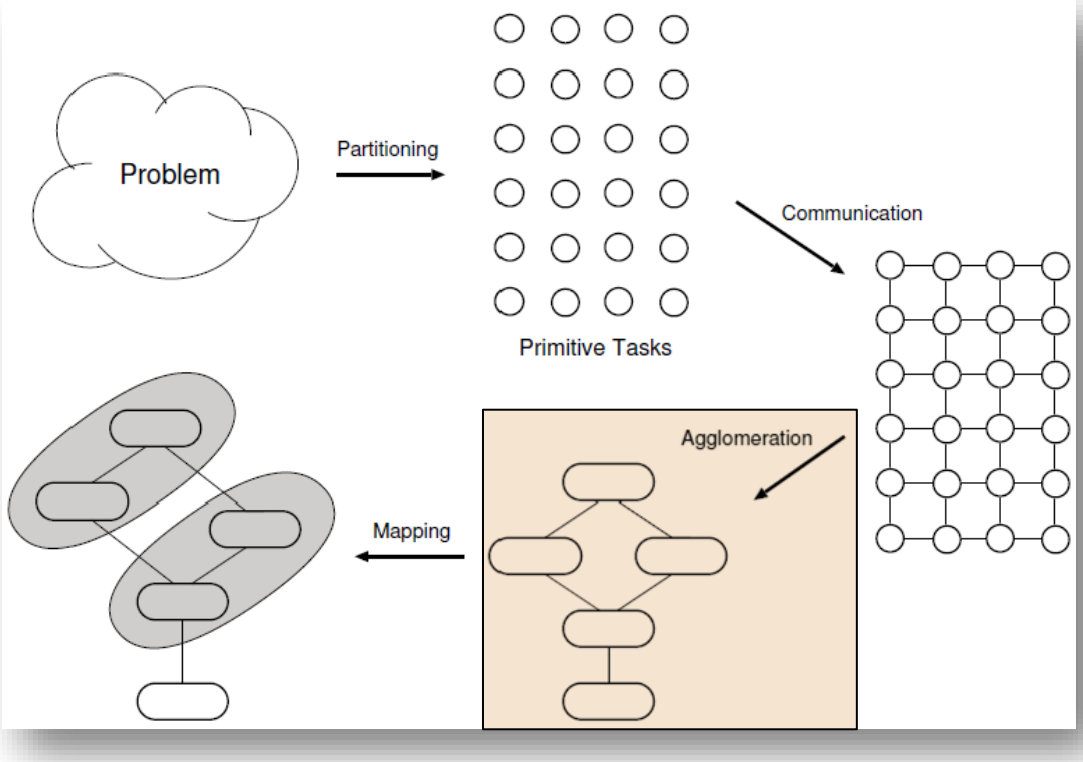
- Communicating with many other tasks adds too much overhead.

✓The **communication** operations should be able to **proceed concurrently.**

- If not, your algorithm is likely to be inefficient and non-scalable to a larger problem instance.

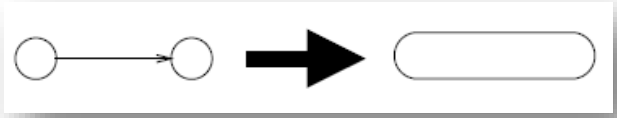
FDM: Agglomeration (1-2)

- Parallelising to the max, introduces (a lot of) overhead, including communication.
- Re-visit partitioning and communication to **reduce overhead.**
- **combine small tasks** into larger ones.
- Agglomeration is an **optimisation problem**.
  - Very often the goals are conflicting, and trade-offs are made.

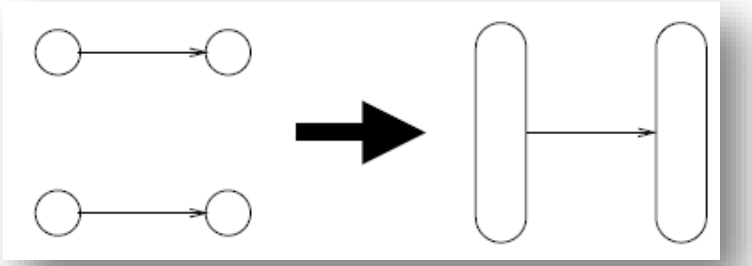


FDM: Agglomeration (2-2)

- Agglomeration tackles granularity.
- Increase Locality
  - Removing overhead by combining two or more tasks.



- Combine groups of tasks that all send and groups of tasks that all receive data from each other.
  - Reduces overhead: time needed for initial start-up.



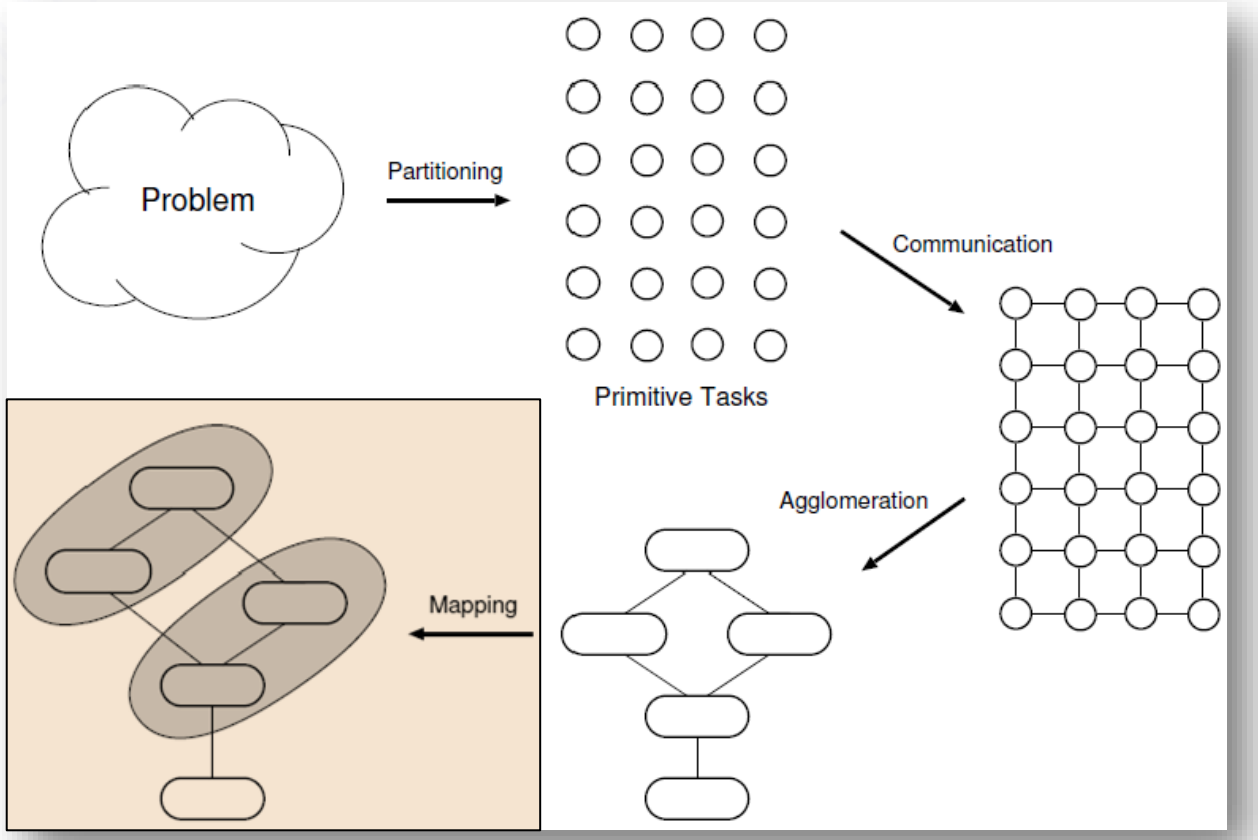
- Too much agglomeration is not a good thing.
  - Must not prevent scalability and other aspects (e.g., code reuse).

Foster's Agglomeration Checklist

- ✓Agglomeration should **reduce communication** costs by increasing locality.
- ✓Agglomeration should produce **tasks** with **similar computation** and **communication** costs.
- ✓The **number** of **tasks** can still **scale** with **problem size**.
- ✓The **trade-off** between the chosen agglomeration and the cost of modification to existing sequential code is **reasonable**.

FDM: Mapping (1-2)

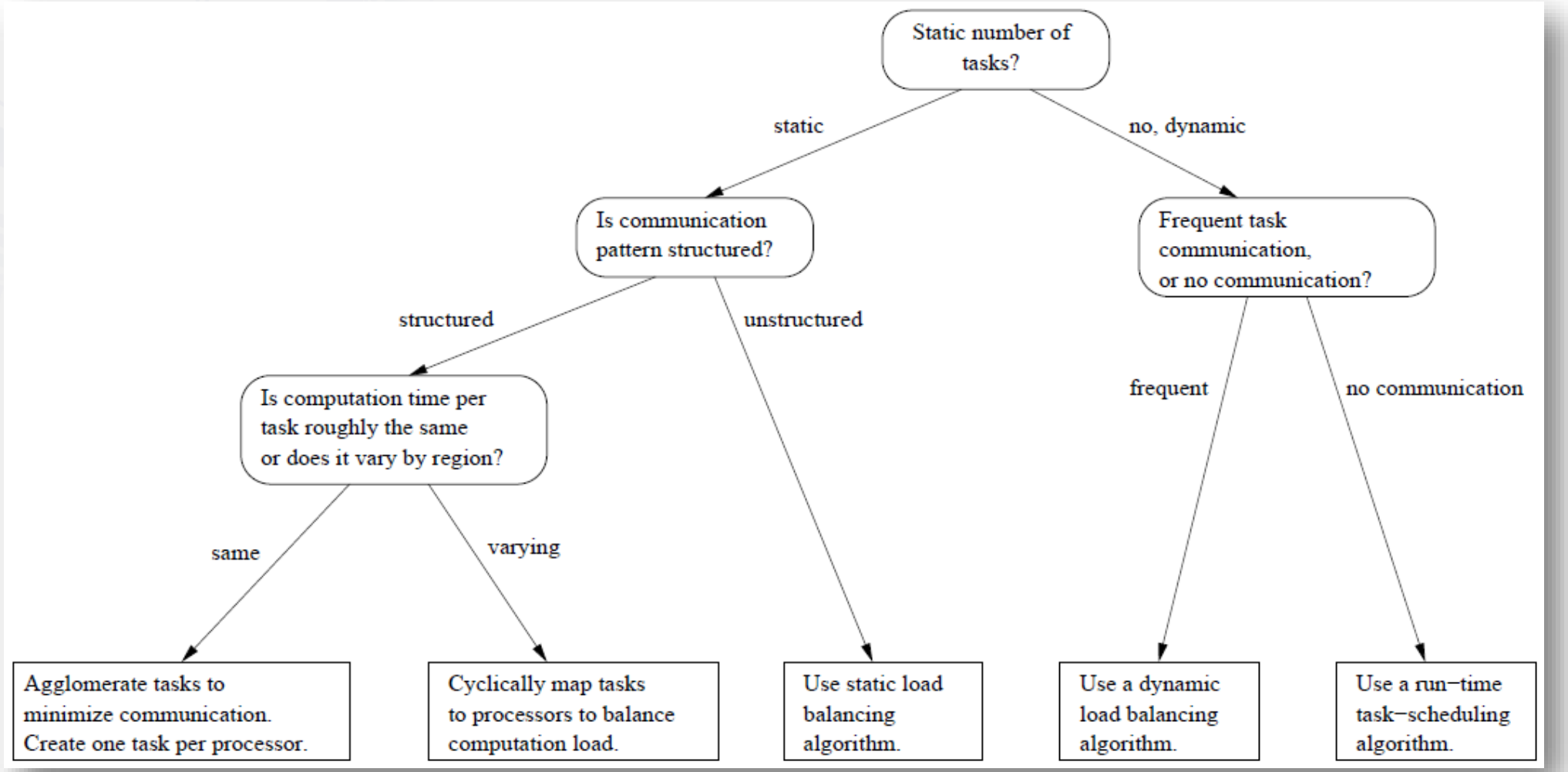
- Assigning processors to tasks
- Goal: **minimise** total **execution** time, & **maximise** processors' **utilisation**.



FDM: Mapping (2-2)

- Approach
  - Tasks that run in parallel should be on different processors.
    - Enhance parallelism.
  - Tasks that communicate frequently should be on the same processor.
    - Increase locality.
- Not an easy thing: the goal is computationally intractable (NP-hard).
- Methods
  - Regular communication: even, cyclic, interleaved assignment.
  - Irregular communication
    - Known in advance: static load balancing
    - Not-known in advance: dynamic load balancing

Deciding a Mapping Strategy



Foster's Mapping Design Checklist

- ✓ Designs based on **one task** per processor and **multiple tasks** per processor have been **considered**.
- ✓ Both **static** and **dynamic allocation** of tasks to processors have been **considered**.
- ✓ If a **centralised load-balancing** scheme is chosen, you have made sure that the manager will **not** become a **bottleneck**.
- ✓ If a **dynamic load-balancing** scheme is chosen, you have **evaluated** the relative **costs** of different **strategies**.

Today, We Are Covering…

- Parallel Algorithms: Intro
- Key Concepts
- Parallel Algorithm Design Methodology
- Common Parallel Algorithm Patterns
- Activity: Designing a Parallel Algorithm

Common Parallel Algorithm Patterns

Data Parallelism

- Divide data into chunks, and perform the same operation on different chunks.
- e.g., matrix multiplication

Task Parallelism

- Divide a task into sub-tasks, apply to same data (could be different data)
- e.g., upload and process an image

Divide & Conquer

- A recursive pattern in which a problem is divided into sub-problems, and result is combined by solving sub-problems.
- e.g., merge sort

Pipeline Parallelism

- computation is divided into stages, and each stage is executed on a different processor
- e.g., compiler (lexical analysis, parsing ,etc.)

Map Reduce

- dividing the problem into two distinct phases: Map (processing data in parallel) and Reduce (aggregating the results)
- commonly used in distributed systems for processing large datasets

Today, We Are Covering…

- Parallel Algorithms: Intro
- Key Concepts
- Parallel Algorithm Design Methodology
- Common Parallel Algorithm Patterns
- Activity: Designing a Parallel Algorithm

Activity: Word Count on a Large Text Dataset

- In Groups (3-4), design a parallel algorithm to solve the following.
- The Problem: Given a large dataset of text (such as books, articles, or logs), count the occurrences of each word in the dataset.
- Questions to consider
  - What is your decomposition strategy?
  - Are there any dependencies?
  - How about communication?
  - What load balancing strategies would you consider?
  - Are there any parallel patterns that would suit this problem?

Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

What Are Your Questions?